Universidad EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Pregrado en Ingeniería de Sistemas

Sistemas Operativos

Parcial 3 – Documento de Estrategia

Profesor

Edison Valencia Díaz

Integrantes

Miguel Colorado Castaño

Sebastián Andrés Ibarra Prada

06 de noviembre de 2026

PARTE I: ANÁLISIS CONCEPTUAL Y PREGUNTAS DIAGNÓSTICAS

1. Mapeo virtual y Page Faults: Cuando un proceso ejecuta mmap(), el kernel verifica que el descriptor del archivo sea válido y que se cumplan los permisos necesarios. También intervienen los mecanismos de seguridad, como el hook LSM mmap_file. Después, se crea o modifica una estructura vm_area_struct dentro del espacio de direcciones del proceso, donde quedan registrados los permisos y las características del mapeo. En esta etapa todavía no se reserva memoria física ni se cargan las páginas en las tablas de páginas. Esto se debe a que Linux utiliza una asignación bajo demanda (lazy allocation). Cuando el proceso intenta acceder por primera vez a una dirección del mapeo, la MMU detecta que la entrada correspondiente no está presente y genera un Page Fault. El kernel busca la VMA que contiene esa dirección y utiliza handle_mm_fault() para resolver el fallo. Si se trata de un archivo convencional, filemap_fault() busca la página en el Page Cache o solicita su lectura desde disco. Finalmente, se instala la entrada correspondiente en la tabla de páginas y se reanuda la ejecución del proceso.

2. MAP_SHARED vs. MAP_PRIVATE: La principal diferencia entre ambos modos se refleja en las banderas de vm_flags dentro de la VMA. Cuando se utiliza MAP_SHARED, se activa VM_SHARED, lo que permite que varios procesos compartan las mismas páginas del archivo en el Page Cache. Por eso, cuando uno de ellos realiza una escritura, los cambios pueden ser observados por los demás procesos y posteriormente persistirse en el archivo. En cambio, con MAP_PRIVATE las modificaciones no afectan el archivo original. Aunque el mapeo permita escritura, las páginas inicialmente compartidas se mantienen protegidas contra escritura. Cuando el proceso intenta modificarlas, se produce un mecanismo de Copy-On-Write (COW), mediante el cual el kernel crea una copia anónima para ese proceso. Desde ese momento, los cambios se realizan sobre esa copia y no sobre el archivo. Desde el punto de vista de seguridad, MAP_SHARED implica un mayor riesgo de modificación del contenido compartido, mientras que MAP_PRIVATE mantiene los cambios aislados. Sin embargo, ambos modos pueden leer inicialmente información proveniente del mismo Page Cache.

3. Mecanismo Copy-On-Write (COW): El mecanismo COW permite que varios procesos compartan inicialmente una misma página de memoria sin necesidad de crear copias desde el principio. En un mapeo privado, la página puede apuntar inicialmente al Page Cache mediante una entrada de solo lectura. Cuando el proceso intenta escribir sobre ella, se produce un fallo de protección y el kernel crea una nueva página anónima, copia su

contenido y actualiza la PTE para que apunte a esa copia con permisos de escritura. También se invalida la entrada correspondiente en la TLB para que la MMU utilice la nueva traducción. Un problema grave ocurre cuando, debido a un error en este procedimiento, la escritura termina afectando la página original en lugar de la copia privada. Esto fue lo que sucedió con Dirty COW (CVE-2016-5195), una vulnerabilidad que involucraba una condición de carrera en get_user_pages() y la gestión de COW. En determinadas circunstancias, la concurrencia con operaciones como madvise(MADV_DONTNEED) permitía que un usuario sin privilegios modificara archivos que normalmente eran de solo lectura, como /etc/passwd o binarios SUID, lo que podía facilitar una escalada de privilegios. En este contexto, VM_MAYWRITE también es importante, ya que controla si un mapeo puede llegar a adquirir permisos de escritura y ayuda a evitar que se habiliten permisos que no corresponden al tipo de acceso autorizado.

4. Intercepción de llamadas en el kernel: Una alternativa para controlar el uso de mmap() sería desarrollar un módulo de kernel (LKM) que modifique la System Call Table. Sin embargo, este enfoque presenta varios problemas en versiones modernas de Linux, especialmente en kernels 5.x y 6.x. Por ejemplo, sys_call_table no suele estar exportada, se encuentra protegida contra escritura y modificarla puede requerir alterar el bit WP del registro CR0. Esto representa un riesgo para la estabilidad del sistema y puede generar problemas de concurrencia. Además, interceptar directamente la syscall implica trabajar con parámetros provenientes del espacio de usuario, lo que puede introducir vulnerabilidades como TOCTOU. Una alternativa más adecuada es utilizar Linux Security Modules (LSM), específicamente el hook mmap_file. Este mecanismo permite intervenir durante la validación de seguridad del mapeo y acceder a información como el archivo involucrado, los permisos solicitados y las banderas utilizadas. Además, permite rechazar operaciones no autorizadas, por ejemplo, devolviendo -EPERM, y puede trabajar junto con mecanismos como SELinux o AppArmor. Aunque requiere que el kernel tenga soporte LSM, es una solución más segura y mantenible que modificar directamente la tabla de llamadas al sistema.

5. Cifrado transparente en memoria: Para que un archivo cifrado en disco pueda utilizarse mediante mmap(), el kernel debe encargarse de descifrar su contenido antes de entregarlo al espacio de usuario. Esto se puede observar en mecanismos como fscrypt y eCryptfs, donde el almacenamiento mantiene los datos cifrados, pero el Page Cache contiene el contenido en texto claro para permitir su lectura y modificación normal. Cuando ocurre un

fallo de página, el kernel solicita la lectura del bloque cifrado y realiza el descifrado antes de que la página quede disponible para el proceso. De manera similar, cuando se modifica una página, los cambios permanecen temporalmente en claro en el Page Cache y se cifran durante el proceso de writeback, antes de escribirse en disco. Es importante tener en cuenta que el cifrado del archivo no garantiza por sí solo la protección del contenido una vez que está descifrado en memoria. Para evitar accesos no autorizados se deben utilizar otros mecanismos del kernel, como los permisos POSIX, las ACL y los controles LSM.

6. Sincronización e integridad (msync y fsync): Cuando un proceso modifica una página compartida mediante mmap(), el hardware y el kernel registran que su contenido ha cambiado, marcándola como sucia (dirty page) dentro del Page Cache. Estas páginas son gestionadas por los mecanismos de writeback del kernel, que se encargan de escribirlas en el almacenamiento de acuerdo con las políticas de memoria y sincronización del sistema. También existen llamadas que permiten solicitar que estos cambios se persistan de forma explícita. msync() permite sincronizar los cambios de una región de memoria mapeada y, cuando se utiliza MS_SYNC, espera a que termine la operación. Por otro lado, fsync() solicita la sincronización de los datos y metadatos correspondientes al archivo, incluyendo los cambios realizados mediante mmap(). Esto tiene una implicación importante en seguridad: un atacante puede modificar el contenido de un archivo mediante un puntero dentro de una región mapeada, sin necesidad de ejecutar write(). Por lo tanto, un mecanismo de seguridad que solamente inspeccione las llamadas write() podría no detectar esas modificaciones. Para controlar este comportamiento es necesario aplicar restricciones en el propio kernel, por ejemplo, mediante un LSM que limite los mapeos compartidos con permisos de escritura sobre archivos sensibles.

7. Control de accesos con eBPF: eBPF permite ejecutar programas dentro del kernel sin necesidad de recompilarlo, siempre que estos cumplan las restricciones del verifier, que comprueba que su ejecución sea segura. Una de sus aplicaciones es la auditoría de operaciones relacionadas con mmap(). Para esto se pueden utilizar mecanismos como kprobes y fexit, que permiten observar funciones del kernel como do_mmap() y registrar información sobre las operaciones realizadas. También es posible utilizar BPF-LSM para intervenir directamente en el control de acceso mediante el hook mmap_file, permitiendo rechazar mapeos que incumplan determinadas políticas de seguridad. Por ejemplo, se podría restringir el uso de PROT_WRITE junto con MAP_SHARED sobre archivos sensibles. Los programas eBPF pueden recopilar información del proceso, como PID, UID

y los parámetros del mapeo, y enviar los eventos a un programa en espacio de usuario mediante un ring buffer. Sin embargo, hay una limitación importante: estos mecanismos permiten observar la creación de los mapeos y determinados fallos de página, pero no registran automáticamente cada lectura o escritura que realiza el proceso una vez que la MMU ya tiene instaladas las entradas correspondientes.

8. Evasión de permisos POSIX: Sí, Linux permite que un proceso abra un archivo con O_RDONLY y posteriormente lo mapee utilizando PROT_READ | PROT_WRITE, siempre que se utilice MAP_PRIVATE y se cumplan las demás condiciones del sistema. Esto ocurre porque el kernel diferencia entre modificar directamente el archivo y modificar una copia privada de su contenido. En los mapeos privados, las escrituras se realizan mediante COW sobre páginas anónimas, por lo que no se modifica el archivo original. Por esa razón, la comprobación del permiso de escritura sobre el descriptor no se aplica de la misma manera que en un mapeo MAP_SHARED. Aunque esto no significa que se estén ignorando los permisos POSIX del archivo, sí puede representar un riesgo desde el punto de vista de seguridad. Un proceso podría modificar temporalmente en su propia memoria información o código que normalmente debería permanecer sin cambios, como las instrucciones de una biblioteca mapeada. Para reducir estos riesgos, se pueden implementar políticas LSM que restrinjan los permisos de escritura o ejecución sobre archivos y mapeos sensibles.

9. Mitigación de ataques de carrera: El problema de las condiciones de carrera aparece cuando dos operaciones concurrentes modifican o consultan una misma región de memoria y el resultado depende del orden en que se ejecutan. En el caso de Dirty COW, la combinación de operaciones como madvise(MADV_DONTNEED) y escrituras forzadas mediante get_user_pages() podía provocar que el kernel manejara incorrectamente una página durante el proceso de COW. Las versiones modernas del kernel incorporan diferentes mecanismos para evitar este tipo de situaciones. Uno de ellos es PageAnonExclusive, que permite identificar cuándo una página anónima es exclusiva y puede tratarse como tal durante determinadas operaciones. También se utilizan mecanismos de sincronización y validación de las entradas de las tablas de páginas para comprobar que una PTE no haya cambiado mientras se resuelve un fallo de escritura. Si las condiciones esperadas ya no se cumplen, el kernel debe repetir la operación correspondiente en lugar de continuar utilizando una referencia que podría ser incorrecta. De esta manera, se busca garantizar que las escrituras de un mapeo privado afecten únicamente a su copia y nunca a la página original compartida del archivo.

10. Aislamiento de memoria y paginación: El aislamiento de memoria depende en gran parte de cómo el kernel administra las páginas físicas y las tablas de páginas de cada proceso. Para esto utiliza estructuras como struct page, que almacena información sobre los marcos de memoria física, y una jerarquía de tablas formada por PGD, P4D, PUD, PMD y PTE, encargada de traducir las direcciones virtuales en direcciones físicas. Gracias a esta organización, cada proceso mantiene su propio espacio de direcciones y solamente comparte memoria con otros procesos cuando el kernel establece las referencias correspondientes. Además, la MMU utiliza diferentes bits de protección para controlar el acceso a la memoria. El bit R/W determina si una página permite escritura, lo que también interviene en la activación de COW; U/S diferencia los accesos de usuario y kernel; y NX impide ejecutar instrucciones desde páginas marcadas como no ejecutables. En este último caso, si un atacante introduce código en una región de datos, como el heap o la pila, no podrá ejecutarlo mientras se mantenga activa la protección NX. También existen mecanismos complementarios como SMEP y SMAP, que restringen determinados accesos del kernel a memoria de usuario. Finalmente, la política W^X busca evitar que una misma región sea escribible y ejecutable al mismo tiempo, y puede reforzarse mediante controles LSM que restrinjan los cambios de permisos realizados sobre los mapeos.

PARTE II: DISEÑO DE LA ESTRATEGIA CON APOYO DE INTELIGENCIA ARTIFICIAL

La estrategia utiliza la IA como apoyo al ciclo de ingeniería del proyecto: analizar la arquitectura, preparar instrucciones de construcción, revisar resultados y documentar decisiones. La IA no se toma como fuente de verdad; las funciones, hooks, opciones de configuración y resultados deben contrastarse con los headers, la documentación y comandos ejecutados en el entorno.

Organización del trabajo

- Claude AI (skill token-efficient): actúa como orquestador y diseñador. Analiza el problema, compara alternativas, define prompts, coordina agentes y revisa los reportes.
- Claude Code (skill anthropic-skills:token-efficient-code): construye código y archivos siguiendo los prompts y restricciones recibidas. No realiza commits, pushes ni merges.
- Sebastián Andrés Ibarra Prada: desarrolla el análisis conceptual y las preguntas diagnósticas de la Parte I.

- Miguel Ángel Colorado Castaño: ejecuta y compila en WSL2 Ubuntu 24.04, aporta capturas y reportes, valida los resultados y realiza los commits y pushes.

Flujo de trabajo

Claude AI explora las estructuras del kernel y plantea alternativas, dejando como pendientes de verificación los hooks, firmas y requisitos de configuración.

Claude AI define prompts con contexto, rol, restricciones y resultado esperado; Claude Code genera los componentes autorizados.

Miguel compila y ejecuta en WSL2, y reporta errores con capturas, observaciones y objetivos concretos.

Claude AI analiza los resultados, identifica posibles causas y propone correcciones puntuales a código, prompts o comandos.

El equipo valida con compilación limpia, pruebas automatizadas y revisión de los logs. Solo se acepta como resultado aquello que tenga evidencia reproducible.

Miguel integra los cambios mediante commits y pushes.

Prompts, verificación y control de alucinaciones

Se documentaron prompts para coordinar la Parte II, definir los roles de exploración arquitectónica, generación y verificación de C, y diseño de pruebas/PoC. También se prepararon instrucciones para el diagrama y la matriz de alternativas. La metodología exige que la IA no afirme que una función existe, que un kernel soporta BPF-LSM o que una prueba pasó sin evidencia verificable.

El README registra como riesgos observados en trabajos anteriores los benchmarks inventados, cifras de pruebas desactualizadas y edición del README equivocado. Para reducirlos, se exige ejecutar comandos reales, revisar archivos y conservar evidencia. Las PoC deben usar archivos temporales y evitar pruebas de escalada de privilegios o la reproducción de Dirty COW.

Estrategia arquitectónica

La alternativa seleccionada es eBPF-LSM (opción C), con intervención en los hooks mmap_file y file_mprotect. La política busca restringir mapeos compartidos con escritura sobre archivos sensibles y auditar eventos mediante un ring buffer con datos del proceso y

del archivo. Una sonda fentry/kprobe se contempla únicamente como apoyo de medición, no como mecanismo principal de control.

Se prefiere esta opción porque permite decidir dentro del kernel sin modificar la System Call Table, sin cargar un módulo LKM propio y con la validación del verifier eBPF. Su viabilidad depende de que el kernel WSL2 tenga BPF_LSM, BTF y BPF habilitados en la lista LSM. Esa condición aparece en el README como un requisito por descubrir y confirmar, no como una capacidad ya demostrada.

PARTE III: IMPLEMENTACIÓN, PRESENTACIÓN Y ARGUMENTACIÓN DE LA SOLUCIÓN

Documento de orden de trabajo para Claude Code (promptConstruccion.md). Define que se termina primero, que lo bloquea de verdad segun reporteBase.md y que se recorta si el contexto o el tiempo se agotan.

Regla: si algo de P2 o P3 compite con P1 por recursos, gana P1. Un subpunto no se da por hecho con texto, se da por hecho con un comando ejecutado. Las cifras salen solo de comandos corridos; lo que no se pueda medir o cargar sin root queda como pendiente con razon Permiso o Manual.

Niveles de prioridad

Nivel	Contenido	Regla
P1	Subpuntos 4.1.1 a 4.1.4 (codigo, demostracion, Makefile, pruebas) y 4.2.a a 4.2.c (justificacion, medicion, limitaciones)	Se terminan primero y con verificacion completa. Es lo que se califica.
P2	Insercion en el README (Parte III)	Solo despues de que cada subpunto P1 tenga estado final.
P3	ReporteConstruccion.md y actualizarReadme.sh	Se hacen al final y cortos. Si P1 quedo incompleto, se registra ahi y no se rellena.

4.1 Entregables de codigo e implementacion

Orden	Subpunto	Entregable	Dependencia	Bloqueo real	Criterio de terminado	Si no alcanza
1 (paralelo con 4.1.2 y 4.1.4)	4.1.1 Codigo C modularizado compatible con kernel 5.x o 6.x	mmapguard/bpf/mmapguard.bpf.c, src/mmapguard.c, src/policy.c/.h, src/events.c/.h	include/mmapguard_common.h y CONTRATO.md del paso 2	Faltan clang, llc, bpftool y libbpf-dev (D7, D8); instalarlos exige sudo con contrasena (D12). Firmas de hooks sin dumpear (D11).	gcc -Wall -Wextra -Werror compila policy.c y events.c sin libbpf; clang -target bpf compila el	Se recorta el fentry opcional sobre handle_mm_fault y el comando events; el

					.bpf.c si existe toolchain.	.bpf.c queda escrito y revisado a mano, pendiente Permiso.
1 (paralelo con 4.1.1 y 4.1.4)	4.1.2 Demostracion de interceptacion de regiones mmap	mmapguard/demo/mg_probe.c y mmapguard/demo/demo.sh	Solo el contrato (CLI y codigos de salida); la parte con defensa necesita 4.1.1 compilado.	Cargar el programa BPF exige root o CAP_BPF (D12); unprivileged_bpf_disabled=2 (D10). Sin sudo no hay demo con defensa activa.	bash demo/demo.sh ejecuta la linea base real (mmap MAP_SHARED con PROT_WRITE) y muestra errno; el paso con defensa queda como instruccion sudo.	Se recorta la variante con mprotect y la salida decorada; se conserva la linea base real y el paso con sudo para Miguel.
2 (despues de 4.1.1)	4.1.3 Makefile sin advertencias	mmapguard/Makefile con targets all, bpf, user, test, clean	4.1.1 terminado; make test llama a tests/run_tests.sh de 4.1.4.	Sin clang el target bpf no puede construir (D7); sin libbpf-dev el loader no enlaza (D8). Se compila solo lo que el toolchain permita.	make clean && make en ~/mg_build con -Wall -Wextra -Werror da 0 advertencias y exit 0; make test corre tests/run_tests.sh.	Se recorta TC_PREFIX para toolchain de usuario y el target bench; el target bpf queda omitido con aviso claro si falta clang.
1 (paralelo con 4.1.1 y 4.1.2)	4.1.4 Scripts de prueba con intentos no autorizados y carreras	mmapguard/tests/run_tests.sh mas pruebas Bash y C de intentos mmap y carrera mmap, mprotect y munmap	policy.c y events.c de 4.1.1 para las pruebas unitarias; el resto solo usa el contrato.	Los casos con defensa cargada requieren root (D12) y quedan marcados REQUIERE ROOT; perf_event_paranoid=2 y sin strace limitan el trazado (D7, D10).	bash tests/run_tests.sh reporta pasaron y fallaron reales, con 0 fallos en lo ejecutable sin root y REQUIERE ROOT en el resto.	Se recorta la repeticion de la carrera (menos iteraciones); se conserva al menos un intento no autorizado y una carrera.

4.2 Argumentacion y analisis de resultados

Orden	Subpunto	Entregable	Dependencia	Bloqueo real	Criterio de terminado	Si no alcanza
5 (primero de 4.2, tras terminar 4.1)	4.2.b Medicion de costos	Tabla en Argumentacion y analisis de resultados del README; bench/ en mmapguard y fragmento en ~/mg_frag/4_2_b.md.	4.1.3 terminado (el Makefile agrega el target bench) y bench_mmap.c compilado en ~/mg_build.	Sin sudo (D12) no se carga BPF: Con defensa queda NO MEDIDO (Permiso). perf_event_paranoid=2 (D10) y strace ausente (D7) limitan herramientas.	Cada celda numerica sale de un comando mostrado; 1000 mmap por corrida, 10 corridas, mediana y rango; entrega bench_con_defensa.sh para Miguel.	Se recorta primero perf stat y se deja solo clock_gettime sobre la linea base; la fila Con defensa sigue como NO MEDIDO.
6 (despues de 4.2.b)	4.2.c Limitaciones y trabajo futuro	Seccion de limitaciones en Argumentacion y analisis de resultados del README; fragmento en ~/mg_frag/4_2_c.md.	Resultados de 4.2.b y filas de ReporteConstruccion.md con razon Limitacion de la solucion escogida.	Limitaciones de D4 y D11: lista LSM activa ilegible y firmas de hooks sin dumpear; marcar POR CONFIRMAR lo no verificado.	Maximo 100 palabras o tabla de hasta 6 filas; incluye decide al crear el mapeo y no por acceso, y depende de bpf en LSM y CAP_BPF.	Se pasa de prosa a tabla de 6 filas y se quita el trabajo futuro, conservando las limitaciones medidas.
7 (despues de 4.2.b y 4.2.c)	4.2.a Justificacion kernel frente a usuario	Subseccion Argumentacion y analisis de resultados del README; fragmento en ~/mg_frag/4_2_a.md que el orquestador inserta.	4.1 terminado y cifras de 4.2.b, para apoyarse en lo medido y en la matriz A a E.	Sin sudo (D12) no hay medicion con defensa; la justificacion no puede citar overhead que no se midio.	Maximo 250 palabras, voz del equipo en primera persona plural; explica por que LD_PRELOAD es eludible y el limite de la PTE.	Se recortan ejemplos y se deja solo el contraste kernel frente a usuario con lo medido; cero cifras nuevas sin comando.

Orden de ejecucion resumido

1) Contrato (mmapguard_common.h y CONTRATO.md). 2) 4.1.1, 4.1.2 y 4.1.4 en paralelo. 3) 4.1.3 cuando termine 4.1.1. 4) 4.2.b, luego 4.2.c, luego 4.2.a. 5) README (P2). 6) ReporteConstruccion.md y actualizarReadme.sh (P3).

Referencias D1 a D12: tabla de pendientes de reporteBase.md, seccion 7.